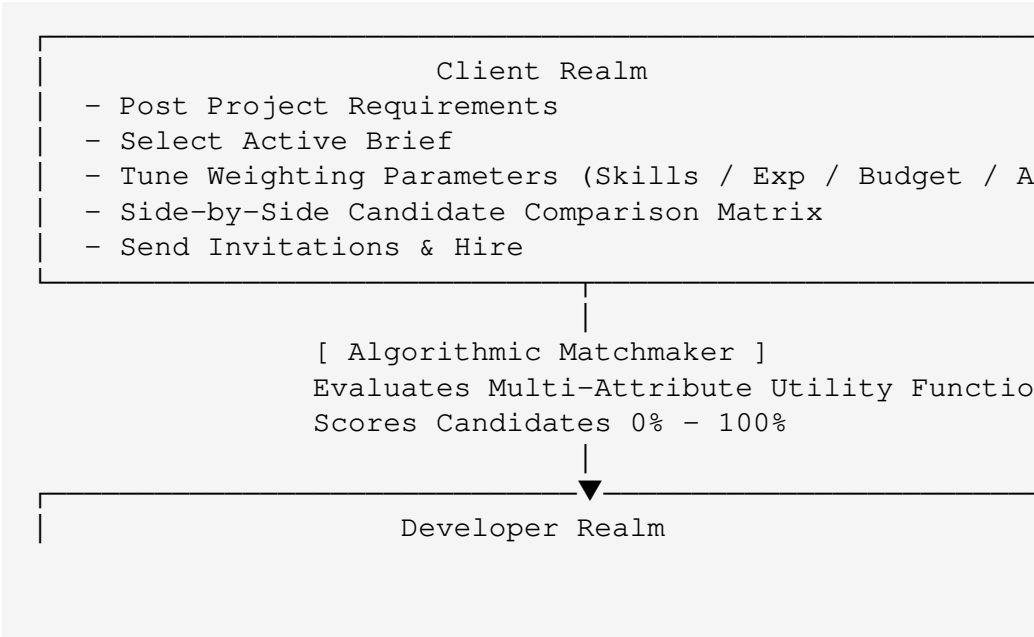
DevMatch — Engineering Architecture & System Specification Manual

Target Audience: Frontend, Backend, Full-Stack, and DevOps Engineers
Version: 1.2.0-Production Spec

1. System Topology & Architecture Overview

DevMatch is a two-sided marketplace designed around automated talent ranking and verified engineering profiles. It eliminates low-effort proposal spam by programmatically matching client technical specifications with developer capabilities.



```
|  - Build Profile (Headline, Rate, Bio, Location)
|  - Technical Skills Matrix (Expert, Advanced, Intermedi
|  - Portfolio Deliverables (GitHub, Live URL, Metrics)
|  - Work Experience & Credentials
|  - Client Opportunity Feed (Live Personal Match Score)
|  - Submit Tailored Proposals
└──────────────────────────────────────────────────────────
```

---

2. The Algorithmic Recommendation Engine

2.1 The Mathematical Model

For any active project requirement P and developer profile D, the composite match score is calculated as:

$$\text{Composite Score} = (S \times W_s) + (E \times W_e) + (B \times W_b) + (A \times W_a)$$

Where the client-tunable weights default to:

- $W_s = 0.45$ (Technical Skills Fit)
- $W_e = 0.25$ (Experience Depth)
- $W_b = 0.20$ (Budget Alignment)
- $W_a = 0.10$ (Availability & Track Record)

2.2 Skill Fit Scoring (S)

1. **Proficiency Multipliers**:
    - `Expert`: 1.00
    - `Advanced`: 0.85
    - `Intermediate`: 0.70
2. **Deliverable Verification Bonus**: $+0.05$ bonus if the required skill appears in the developer's verified portfolio project tech stacks.

3. **Ecosystem Fuzzy Overlap**: 0.40 partial credit for adjacent technologies (e.g., React ↔ Next.js, Go ↔ Systems, Kubernetes ↔ Docker, SQL ↔ PostgreSQL).

2.3 Seniority & Budget Curves (E & B)

- **Experience**: $$\text{Score} = \begin{cases} 100% & \text{if } \text{Tenure} \ge \text{ReqYears} \ \max\left(30%, \frac{\text{Tenure}}{\text{ReqYears}} \times 100\right) & \text{otherwise} \end{cases}$$
- **Budget**:
  - Within $[B_{\min}, B_{\max}]$: $100%$
  - Below $B_{\min}$: $95%$ (high cost-efficiency)
  - Above $B_{\max}$: $\max(20%, 100 - (\text{OvershootRatio} \times 130))$

---

3. Frontend Architecture (React 19 + TypeScript)

Component Tree

```
src/
├── App.tsx                     # Global Root, routing, r
├── components/
│   ├── Navbar.tsx              # Top Bar Contract (Wordm
│   ├── ClientView/
│   │   ├── MatchmakerBar.tsx   # Brief switcher, brief s
│   │   ├── DeveloperCard.tsx   # Unboxed metadata, match
│   │   ├── DeveloperDetailModal.tsx # Full bio, skills m
│   │   ├── CompareDrawer.tsx   # Multi-candidate compari
│   │   ├── ProjectPostModal.tsx# Form for client project
│   │   └── ProjectBriefsView.tsx # Overview of client po
│   ├── DeveloperView/
```

```
│   │   ├── DeveloperProfileEditor.tsx # Skills, projects
│   │   └── DeveloperProjectsFeed.tsx  # Project opportun
│   └── Shared/
│       ├── DocViewerModal.tsx  # Printable in-app archit
│       ├── MessagesModal.tsx   # Direct chat messaging b
│       ├── ProposalsModal.tsx  # Bid review, accept/hire
│       └── AuthModal.tsx       # Account creation & quic
├── services/
│   ├── matchingAlgorithm.ts    # Pure TS recommendation
│   ├── pdfGenerator.ts         # Client-side multi-page
│   └── storage.ts              # LocalStorage persistenc
└── types/
    └── index.ts                # TypeScript domain model
```

---

4. Backend Architecture & Relational Database Schema

PostgreSQL Relational DDL

```
-- Users
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    role VARCHAR(20) NOT NULL CHECK (role IN ('client', '
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    company VARCHAR(150),
    title VARCHAR(150),
    avatar_url TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Developer Profiles
CREATE TABLE developer_profiles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    hourly_rate NUMERIC(10, 2) NOT NULL,
    years_experience INT NOT NULL,
    availability VARCHAR(50) NOT NULL,
    primary_category VARCHAR(100) NOT NULL,
    location VARCHAR(100),
    timezone VARCHAR(50),
    bio TEXT,
    rating NUMERIC(3, 2) DEFAULT 5.00,
    reviews_count INT DEFAULT 0
);

-- Skills Matrix
CREATE TABLE developer_skills (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    developer_id UUID REFERENCES developer_profiles(id) O
    skill_name VARCHAR(100) NOT NULL,
    proficiency_level VARCHAR(30) CHECK (proficiency_leve
    years_experience INT NOT NULL
);

-- Portfolio Projects
CREATE TABLE portfolio_projects (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    developer_id UUID REFERENCES developer_profiles(id) O
    title VARCHAR(200) NOT NULL,
    role VARCHAR(100) NOT NULL,
    description TEXT NOT NULL,
    tech_stack TEXT[] NOT NULL,
    live_url TEXT,
    github_url TEXT,
    metrics TEXT,
    completed_year VARCHAR(10)
);

-- Project Requirements
CREATE TABLE project_requirements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    client_id UUID REFERENCES users(id) ON DELETE CASCADE
    title VARCHAR(200) NOT NULL,
```

```
    description TEXT NOT NULL,
    category VARCHAR(100) NOT NULL,
    required_skills TEXT[] NOT NULL,
    preferred_experience_years INT NOT NULL,
    budget_min NUMERIC(10, 2) NOT NULL,
    budget_max NUMERIC(10, 2) NOT NULL,
    duration VARCHAR(50) NOT NULL,
    status VARCHAR(30) DEFAULT 'open'
);

-- Proposals & Bids
CREATE TABLE proposals (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID REFERENCES project_requirements(id) O
    developer_id UUID REFERENCES developer_profiles(id) O
    client_id UUID REFERENCES users(id) ON DELETE CASCADE
    status VARCHAR(20) DEFAULT 'pending' CHECK (status IN
    proposed_rate NUMERIC(10, 2) NOT NULL,
    cover_letter TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

---

5. RESTful API Contract

Method	Endpoint	Description
`GET`	`/api/v1/talent/match`	Returns ranked developer profiles with match breakdown against `projectId`
`POST`	`/api/v1/projects`	Creates a new client project brief
`GET`	`/api/v1/projects`	Lists active project briefs
`PUT`	`/api/v1/developers/profile`	Updates developer bio, hourly rate, and availability

POST	/api/v1/developers/skills	Upserts skills with proficiency tiers
POST	/api/v1/developers/projects	Adds verified portfolio project with GitHub and live links
POST	/api/v1/proposals	Submits a proposal / invitation
PATCH	/api/v1/proposals/:id	Updates status (accepted | declined)
GET	/api/v1/messages	Fetches conversation thread
POST	/api/v1/messages	Transmits direct message